

جامعة دمشق

كلية الهندسة المعلوماتية

قسم هندسة البرمجيات و نظم المعلومات



IR-System

طلاب المجموعة:

حسان محمد غنام

عبدالرحمن يحيى حوير

غيث حمزه أبو راشد

الاء عيسى سليمان

إشراف:

م.سليمي المحايري

● الملخص التنفيذي (Executive Summary):

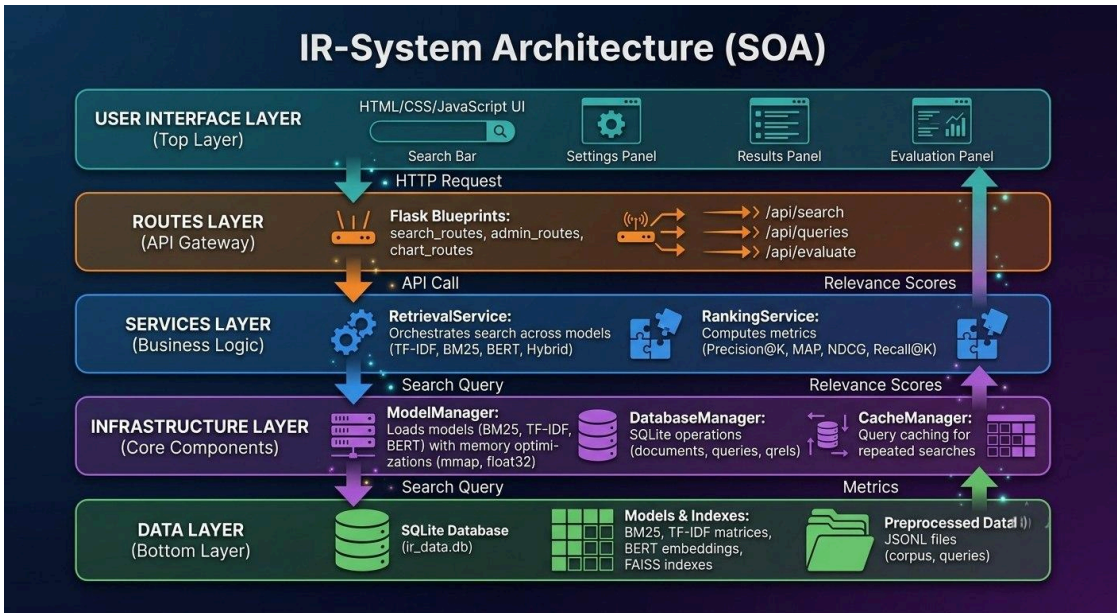
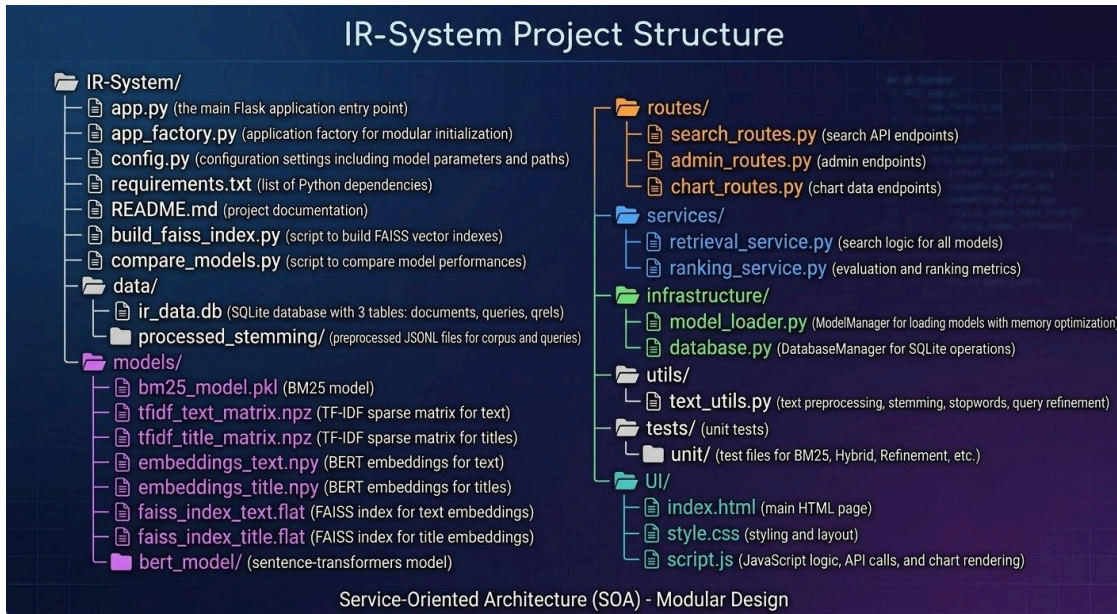
يقدم هذا التقرير مشروعاً متكاملًا لنظام استرجاع المعلومات (IR System) يهدف إلى بناء محرك بحث نصي قادر على استرجاع الوثائق الأكثر صلة باستعلامات المستخدم.

تم تطبيق وتقييم أربعة نماذج رئيسية:

- TF-IDF (مع دمج العناوين).
- BM25 (مع ضبط المعاملات).
- BERT (التضمينات الدلالية مع دمج الحقول).
- HYBRID (مع Parallel & Serial).

أظهرت النتائج تفوق نموذج BM25 بعد ضبط معاملاته ($k_1=2.0$, $b=0.60$) محققاً $\text{MAP}@10 = 0.1470$ و $\text{Precision}@10 = 76.12\%$.

تم تطبيق بنية موجهة للخدمات (SOA) مع تحسينات متقدمة في الذاكرة (mmap, float32, cache) وتكامل مع FAISS لتسريع البحث الدلالي.



● جدول المحتويات (Table of Contents):

1. مقدمة واختيار مجموعة البيانات.
2. بيئة العمل والأدوات المستخدمة.
3. معالجة البيانات (Preprocessing).
4. تمثيل النماذج (Models Representation).
5. الفهارس (Indexing).
6. معالجة الاستعلامات وتحسينها.
7. المطابقة وترتيب النتائج.
8. البنية التحتية والهندسة المعمارية (SOA).
9. تقييم النظام والنتائج.
10. واجهة المستخدم (UI).
11. الميزة الإضافية (USE VECTOR STORE FAISS).
12. التحديات التي تم حلها.
13. تقسيم العمل (Team Work Distribution).
14. المصادر والمراجع.
15. روابط المشروع.
16. الخاتمة والتوصيات.
17. الصور ولقطات الشاشة للواجهات الرسومية.

1. مقدمة واختيار مجموعة البيانات:

- 1.1. نظرة عامة:
يُعد استرجاع المعلومات (Information Retrieval) أحد الركائز الأساسية في معالجة اللغة الطبيعية، ويهدف إلى استخراج الوثائق الأكثر صلة باستعلام المستخدم.
في هذا المشروع قمنا ببناء نظام IR متكامل لتقييم ومقارنة أداء منهجيات بحث مختلفة (VSM_TF-IDF) التقليدية، (BM25) الاحتمالية، (BERT Embeddings) الدلالية، (HYBRID) الهجينة على مجموعة بيانات متخصصة في استرجاع الحجج.
- 1.2. اختيار مجموعة البيانات (Dataset Selection):
تم اختيار مجموعة beir/webis-touche2020 لعدة أسباب:
التخصص: المجموعة مخصصة لمهمة استرجاع الحجج (Argument Retrieval) حيث يحتوي كل استعلام على سؤال جذلي وتحتوي الوثائق على حجج مؤيدة أو معارضة.
الحجم المناسب: تحتوي على 382,545 وثيقة، مما يوفر تحدياً حقيقياً للنماذج من حيث زمن البحث وكفاءة الفهرسة، مع إمكانية تشغيلها على أجهزة محلية متوسطة الإمكانيات بعد التحسين.
التقييم الجاهز: توفر المجموعة 49 استعلاماً و 2,962 ملف تقييم مما يتيح تقييماً موضوعياً ودقيقاً لمقاييس MAP و NDCG.
- 1.3. خصائص مجموعة البيانات (Dataset Characteristics):
طبيعة البيانات: نصية بالكامل وباللغة الإنجليزية.
هيكل الوثيقة (Doc): م doc_id, title, text, stance, url.
هيكل الاستعلام (Query): م query_id, text, description, narrative.

2. بيئة العمل والأدوات المستخدمة (Development Environment):

- تم تصميم النظام وتطويره واختباره باستخدام بيئة تطوير هجينة (Hybrid Development Environment) تجمع بين قوة الحوسبة السحابية وسرعة الوصول إلى البيانات محلياً.
- 2.1. البنية التحتية للأجهزة والحوسبة:
التطوير المحلي (Local Machine):
تشغيل خادم Flask، بناء واجهة المستخدم، اختبار النماذج الخفيفة، وإدارة قاعدة البيانات.
الحوسبة السحابية (Google Colab):
تدريب TF-IDF، توليد تضمينات BERT (استغرق ~9 ساعات)، تدريب BM25، وضغط الملفات الثقيلة الاحتفاظ بها محلياً.
التخزين: تم استخدام Google Drive لنقل النماذج والملفات الوسيطة.
 - 2.2. لغات البرمجة والمكتبات الأساسية:
تم بناء النظام باستخدام Python 3.10+، مع الاعتماد على مكتبات مفتوحة المصدر:
خادم الويب (Backend & Frontend):
تم استخدام إطار العمل Flask بهدف بناء واجهة برمجة التطبيقات (API) الخلفية.
معالجة النصوص (Preprocessing):
اعتمدنا على مكتبة NLTK (الإصدار 3.8.1) إلى جانب وحدة unicodedata؛ لإجراء عمليات التقطيع (Tokenization)، وإزالة الكلمات الشائعة (Stopwords)، واستخراج جذور الكلمات (Stemming) لتحسين جودة النصوص المدخلة.
النماذج التقليدية (Traditional Information Retrieval):
لبناء أنظمة الاسترجاع الكلاسيكية، تم توظيف scikit-learn (خاصةً TfidfVectorizer) لبناء نموذج فضاء المتجهات (VSM)، ومكتبة rank-bm25 (خاصةً BM25Okapi) لتنفيذ نموذج الاحتمالات BM25.
 - النماذج الدلالية (Semantic Models):

وللتضمنيات الدلالية العميقة، تم استخدام مكتبة sentence-transformers مع النموذج المُدرَّب مسبقاً all-MiniLM-L6-v2؛ لتوليد تمثيلات متجهية (Embeddings) تعبر عن المعنى السياقي للنصوص.

الحسابات الرقمية والجبر الخطي (Numerical Computations):
تم التعامل مع المصفوفات والمتجهات باستخدام NumPy، إلى جانب SciPy (خاصة المصفوفات المتناثرة (sparse)؛ لحساب مقاييس التشابه (مثل Cosine) وإجراء عمليات ضرب النقطي (Dot Product) بكفاءة.

الفهرسة السريعة (Fast Nearest Neighbor Search):
لتسريع عملية البحث عن أقرب الجيران في فضاء التضمنيات، تم بناء فهرس متجهات باستخدام مكتبة FAISS (الإصدار المخصص لوحدة المعالجة المركزية faiss-cpu)؛ مما قلل زمن الاستعلامات الدلالية بشكل ملحوظ.

قاعدة البيانات والتخزين المحلي (Database & Persistence):
استُخدمت SQLite3 لتخزين واسترجاع الوثائق وبيانات الاستعلامات هيكلياً، بالإضافة إلى مكتبتَي json و pickle لحفظ الكائنات والمصفوفات الوسيطة بشكل محلي لضمان إعادة الاستخدام دون تكرار المعالجة.

إدارة البيانات وعرض النتائج (Data Management & Visualization):
أضيفت مكتبة tqdm لعرض أشرطة التقدم أثناء المعالجات الطويلة، ومكتبة tabulate لعرض الجداول المقارنة للنتائج بشكل منظم داخل مخرجات التقرير.

- 2.3. أدوات إدارة المشروع والتحكم بالإصدارات GitHub لإدارة إصدارات الأكواد.
البيئات الافتراضية (venv): لعزل مكتبات المشروع ومنع التعارض.
هيكلية المجلدات: اتباع نمط هندسي نظيف (Clean Architecture) يفصل بين البيانات، النماذج، المنطق، والواجهة (SOA).
2.4. التحديات الهندسية التي تم التغلب عليها:
سرعة البحث في BERT: تم حلها باستخدام FAISS وتطبيق المتجهات مسبقاً، مما قلل زمن البحث إلى ~1.5 ثانية.
توافق NLTK: تم تثبيت الحزم (punkt, stopwords, punkt_tab) يدوياً لضمان العمل دون اتصال بالإنترنت.

3. معالجة البيانات (Preprocessing):

لضمان جودة المطابقة بين الاستعلام والوثائق، تم تطبيق خطوات تنظيف موحدة على كل من النصوص والعناوين، مع اعتماد نهج Stemming:

- 3.1. خطوات التنظيف الأساسية:
التطبيع (Normalization): توحيد ترميز يونيكود (NFKC).
تحويل الحروف: جعل جميع الأحرف صغيرة (Lowercasing).
إزالة علامات الترقيم: باستخدام string.punctuation.
التقطيع (Tokenization): باستخدام nltk.word_tokenize.
إزالة كلمات التوقف (Stopwords): باستخدام قائمة NLTK الإنجليزية.
الاشتقاق (Stemming): باستخدام PorterStemmer (تم اعتماده بعد المقارنة مع Lemmatization).
3.2. المقارنة النهائية (Stemming vs Lemmatization)
تم تنفيذ المعالجة مرتين (لكل من العنوان والنص) وحفظ المخرجات في مجلدين منفصلين. أظهرت التجارب أن:
Stemming تفوق على Lemmatization في نموذج TF-IDF المدمج، محققاً تحسناً بنسبة ~1.8% في MAP و ~4% في NDCG.
السبب: Stemming يدمج المرادفات الصرفية (مثل "tenure" و "tenured") بكفاءة أعلى في السياق الجدلي.
القرار: تم اعتماد Stemming كمعيار نهائي لجميع النماذج.
3.3. دمج الحقول (Field-Weighted Modeling)
تم تطبيق دمج الحقول (العنوان والنص) BERT باستخدام المعادلة:
$$(\text{Final_Score} = (\alpha \times \text{Score_Title}) + ((1-\alpha) \times \text{Score_Text}))$$

حيث تم اعتماد $\alpha = 0.4$ كقيمة مثلى أي (40% عنوان، 60% نص).

4. تمثيل النماذج (Models Representation):

تم تنفيذ أربعة تمثيلات رئيسية لتغطية طيف واسع من تقنيات IR:

4.1 VSM_TF-IDF (نموذج الفضاء المتجهي):

التنفيذ: TfidfVectorizer من scikit-learn.

الفهرسة: يتم تحميل مصفوفات npz. والمفردات pkl. من مجلد ./models.

4.2 BM25 (النموذج الاحتمالي - الأفضل):

التنفيذ: مكتبة rank_bm25 (نموذج BM25Okapi).

ضبط المعاملات (Hyperparameter Tuning):

تم اختبار 66 تركيباً لقيم (k1, b) على جميع الاستعلامات الـ 49.

القيم الفعلية المعتمدة في النظام: k1=2.0, b=0.60 (موتقة في config.py).

(ملاحظة: يحتوي ملف الاختبارات 2.0/0.60 (test_bm25_params.py).

النتائج: حقق k1=2.0, b=0.60 أعلى MAP@10 = 0.1470 و Precision@10 = 76.12%.

التخزين: bm25_model.pkl (حجم 376 ميجابايت).

4.3 BERT Embeddings (النموذج الدلالي):

التنفيذ: all-MiniLM-L6-v2 من Sentence-Transformers، حجم المتجه 384.

التوليد: تم في كولا ب (استغرق 9 ساعات) وحفظه في embeddings_text.npy و embeddings_title.npy.

(587 ميجابايت لكل ملف).

التطبيق: L2 Normalization لتسريع الحساب بالضرب النقطي المباشر.

النتائج: حقق MAP@10 = 0.0385 و Precision@10 = 30.41%.

4.4 النماذج الهجينة (Hybrid Models):

4.4.1 الهجين المتوازي (Parallel Hybrid):

الآلية: دمج مرجح (Weighted Sum) لدرجات TF-IDF، BM25، BERT.

التجربة: تم اختبار 21 تركيباً من الأوزان.

أفضل الأوزان: BERT(0.00) + BM25(0.80) + TF-IDF(0.20) حقق MAP@10 = 0.1366.

الخلاصة: لم يقدم تحسناً جوهرياً على BM25 المنفرد.

4.4.2 الهجين التسلسلي (Serial Hybrid):

الآلية: استرجاع أولي (Initial-K=100) باستخدام نموذج أول، ثم إعادة ترتيب النتائج باستخدام نموذج ثاني.

التجربة: تم اختبار جميع تبديل النماذج الثلاثة (15 تسلسلاً).

أفضل تسلسل: BM25 → TF-IDF حقق MAP@10 = 0.1264.

الخلاصة: هذا يثبت أن BM25 وحده كافٍ في هذه المجموعة.

5. الفهارس (Indexing):

تم تصميم نظام فهرسة متعدد المستويات لدعم آليات الاسترجاع المختلفة، وذلك على النحو التالي:

5.1 الفهرس المقلوب (Inverted Index):

تم بناء هذا النوع باستخدام مكتبة `rank\_bm25`، حيث تم حفظ نموذج BM25 الناتج في ملف

`bm25\_model.pkl` (وهو موجود فعلياً في النظام). يُستخدم هذا الفهرس خصيصاً لتشغيل خوارزمية البحث

الاحتمالية BM25، والتي تعتمد على حساب أوزان الكلمات استناداً إلى تكرارها داخل الوثيقة ومعكوس تكرار

ظهورها في المجموعة الكلية.

5.2 الفهرس المتجهي (Vector Space Model - VSM):

اعتمدنا في هذا النوع على المصفوفات المتناثرة من نوع `csr\_matrix` (المقدمة من مكتبة `SciPy`) لتخزين

أوزان TF-IDF. وقد صدرنا هذا الفهرس على هيئة ملفين؛ الأول يحمل المتجهات المضغوطة بصيغة `npz`،

- والثاني يحتوي على كائن المحول (`TfidfVectorizer`) بصيغة `pickle` (وكلاهما موجودان فعلياً). يُستعمل هذا الفهرس لتنفيذ البحث القائم على التشابه (Cosine Similarity) في فضاء المتجهات التقليدي.
- 5.3. الفهرس الدلالي (Semantic Embeddings Index):
لتوليد التضمينات الدلالية العميقة، تم استخدام مصفوفات `NumPy` لتخزين المتجهات المُنتجة من نموذج BERT. هذه الملفات ذات الامتداد `npz` متاحة وجاهزة في النظام، وتُستخدم مباشرة في عمليات البحث الدلالي التي تعتمد على تضمينات الجُمْل لفهم المعنى السياقي للنصوص.
- 5.4. الفهرس العلائقي (Relational Database Index):
إلى جانب الفهارس السابقة، تم توظيف قاعدة بيانات `SQLite` كطبقة تخزين علائقية يُستخدم هذا الفهرس بشكل أساسي لربط المستندات بالنصوص الأصلية، وعرض محتوى الوثائق بشكل منسق، وكذلك لتخزين بيانات التقييم والمقارنة بين النماذج المختلفة.
- 5.5. قاعدة البيانات (SQLite):
تُعد قاعدة البيانات المحلية `ir\_data.db` العمود الفقري لإدارة البيانات الأولية والنتائج في هذا المشروع. وتحتوي الأرقام والإحصائيات الفعلية التالية (بناءً على محتويات الملفات المصدرية):
جدول المستندات (`documents`):
يضم هذا الجدول 382,545 سجلاً، تم استيرادها مباشرةً من الملف المصدر `corpus\_original.jsonl`، حيث يمثل كل سجل وثيقة نصية فريدة ضمن المجموعة المستخدمة.
جدول الاستعلامات (`queries`):
يحتوي على 49 استعلاماً بحثياً، وقد تم تغذية هذا الجدول من الملف المصدر `queries\_cleaned.jsonl`، ويمثل مجموعة الأسئلة أو العبارات التي سيتم تقييم أداء نظام الاسترجاع بناءً عليها.
جدول تقييم الملاءمة (`qrels`):
يتضمن 2,962 سجلاً للتقييم، مستوردة من الملف `test.qrels`. يحدد هذا الجدول علاقة الصلة (Relevance) بين أزواج (الاستعلام - الوثيقة)، ويُستخدم كمرجع لحساب مقاييس الدقة مثل Mean Average Precision (MAP) أو Recall@k.

6. معالجة الاستعلامات وتحسينها (Query Processing & Refinement)

- 6.1. معالجة الاستعلام الأساسية تطبق نفس خطوات التنظيف (Stemming، إزالة Stopwords) على استعلام المستخدم لضمان التوافق في الفضاء المتجهي، باستخدام دالة `clean\_query` في `text\_utils.py`.
- 6.2. تحسين الاستعلام (Query Refinement)
تم اختبار نوعين من التحسين في `text\_utils.py` (مع واجهة `refineToggle`):
الأولى بالتصحيح الإملائي وإضافة المرادفات، والتي أظهرت انخفاضاً حاداً في MAP بسبب التشويش الدلالي (تم تجاهلها) والثانية بالتصحيح الإملائي فقط، وكان تأثيرها شبه محايد (تم اعتمادها بالمشروع).

7. المطابقة وترتيب النتائج (Matching & Ranking):

- تم اعتماد أربع آليات لمطابقة الاستعلامات مع الوثائق وترتيب النتائج:
- 7.1. جيب التمام (Cosine Similarity): طُبِّق على متجهات TF-IDF و BERT بعد تطبيعها مسبقاً، باستخدام الضرب النقطي المباشر لحساب التشابه.
- 7.2. درجة (BM25 Score): طُبِّق نموذج BM25 الاحتمالي مع معاملات k_1 و b ، مع معايرة طول الوثيقة.
- 7.3. الدمج المتوازي (Weighted CombSUM): جمع درجات النماذج المتعددة بأوزان مرجحة.
- 7.4. الدمج التسلسلي (RRF): دمج النتائج بناءً على المراتب (Ranks) بدلاً من الدرجات، باستخدام خوارزمية Reciprocal Rank Fusion.

8. البنية التحتية والهندسة المعمارية (SOA):

- تم اعتماد بنية موجهة للخدمات (Service-Oriented Architecture) موثقة في `app\_factory.py` و `routes/` و `services/`:
8.1. الطبقات المعمارية:

- 8.1.1 طبقة الواجهة : UI/index.html, UI/style.css, UI/script.js
- 8.1.2 طبقة التوجيه : routes/search_routes.py, routes/admin_routes.py, routes/chart_routes.py
- 8.1.3 طبقة الخدمات (Services): RetrievalService : منطق البحث (services/retrieval_service.py).
- 8.1.3.1 RankingService : التقييم وإعادة الترتيب (services/ranking_service.py).
- 8.1.4 طبقة البنية التحتية (Infrastructure): ModelManager : تحميل النماذج مع تحسينات الذاكرة (mmap, float32) في infrastructure/model_loader.py
- 8.1.4.1 DatabaseManager : التعامل مع SQLite في infrastructure/database.py
- 8.2 استراتيجيات تحميل النماذج وتحسين الذاكرة

التحميل: يتم تحميل النماذج فوراً عند بدء التشغيل (Eager Loading) لضمان جاهزية النظام الكاملة (كما هو موضح في config.py مع LOAD_*_ON_START = True).
تحسين الذاكرة: تم استخدام mmap لتحميل الملفات الكبيرة، و float32 لتقليل حجم المصفوفات.

9. تقييم النظام والنتائج (Evaluation & Results)

- 9.1 المقاييس المعتمدة: يتم حساب المقاييس في ranking_service.py:
Precision@K: نسبة الوثائق ذات الصلة في أول K نتيجة.
MAP (Mean Average Precision): متوسط الدقة عبر جميع الاستعلامات.
NDCG (Normalized Discounted Cumulative Gain): يقيس جودة الترتيب مع مراعاة درجات الصلة (2-0).
Recall@K: نسبة الوثائق ذات الصلة المسترجعة.

- 9.2 تم تقييم أداء النماذج الخمسة على مجموعة الاستعلامات البالغ عددها 49 استعمالاً، مع اعتماد مقياس Top-10 للمقارنة. أظهر نموذج BM25 (بعد ضبط المعاملات: $k1=2.0$ و $b=0.60$) أداءً متميزاً مقارنةً بباقي النماذج، حيث حقق أعلى القيم في جميع المقاييس: MAP@10 (0.1470)، Precision@10 (76.12%)، و NDCG@10 (0.4592)، مع زمن استجابة مقبول. في المقابل، سجل نموذج TF-IDF كخط أساس (Baseline) أداءً متواضعاً بـ (42.24%، 0.0624، 0.2387). أما نموذج BERT (مع دمج العناوين بمعامل ترجيح $\alpha=0.4$) فكان الأسرع بين النماذج، لكنه جاء الأسوأ أداءً بـ (30.41%، 0.0385، 0.1611). بالنسبة للنماذج الهجينة، سجل النموذج المتوازي (Parallel Hybrid) قيمةً (73.47%، 0.1366، 0.4211)، بينما سجل النموذج التسلسلي (Serial Hybrid) قيمةً (68.37%، 0.1264، 0.4091).
- 9.3 يُعد نموذج BM25 النموذج الأفضل من حيث التوازن بين السرعة والدقة، حيث تفوق على جميع النماذج الأخرى في مقاييس الدقة مع زمن استجابة معقول. أما نموذج BERT، فـ رغم سرعته الكبيرة، فقد جاء الأدنى أداءً.

10. واجهة المستخدم (User Interface):

- لتسهيل تفاعل المستخدم مع نظام الاسترجاع وتوفير تجربة بحث مرنة تسمح بتجربة الإعدادات المختلفة وعرض النتائج بشكل منظم تم تطوير واجهة أمامية تفاعلية (Frontend Interface). تعتمد الواجهة على ثلاث صفحات رئيسية: `index.html` لهيكل العناصر، و `script.js` للتحكم في التفاعلات والمنطق البرمجي، بالإضافة إلى الاتصال بواجهة برمجة التطبيقات الخلفية (`api/queries/` وغيرها).
- 10.1 مكونات الواجهة: تتوزع مكونات الواجهة الرئيسية على أربعة أقسام وظيفية:
 - 10.1.1 شريط البحث العلوي: يحتوي على حقل لإدخال النص (Input Field) لإدخال الاستعلامات بحرية بالإضافة إلى قائمة منسدلة (Dropdown) تحتوي على الاستعلامات المحفوظة مسبقاً والتي يتم جلبها عبر `api/queries/` لتسهيل اختيار الاستعلامات المعيارية وإعادة تشغيل التجارب.
 - 10.1.2 لوحة الإعدادات (اليسار): توفر للمستخدم خيارات التحكم في عملية البحث وتشمل:
 - اختيار النموذج (`model`) من بين النماذج المتاحة (TF-IDF، BM25، BERT، Hybrid).

- التحكم في عدد النتائج المسترجعة (`top_k``) لتحديد حجم قائمة النتائج المعروضة.
 - زر تفعيل/إلغاء التصحيح الإملائي (`refineToggle``) للتحكم في تطبيق استراتيجيات التصحيح الإملائي على الاستعلام.
 - زر تفعيل/إلغاء فهرس (`faissToggle``) لتسريع البحث الدلالي.
- 10.1.3. لوحة النتائج (الوسط): تعرض الوثائق المسترجعة بترتيب تنازلي حسب درجة الملاءمة (Relevance Score) مع عرض المعلومات الأساسية لكل وثيقة (كالمعرف، العنوان، ومقتطف من النص) لتسهيل تقييم المستخدم للنتائج.
- 10.1.4. لوحة التقييم (اليمين): تظهر عند تفعيل زر التقييم (`evalToggle``) وتقوم بعرض مقاييس الأداء مما يتيح للمستخدم مقارنة أداء النماذج المختلفة بشكل لحظي وبصري لكل استعلام، وتعرض الوثائق الملائمة ولم يتم عرضها بالنتائج.

11. الميزة الإضافية:

- استخدام (FAISS - Vector Store)
- 11.1. سكربت البناء: `scripts/build_faiss_index.py` يستخدم `faiss.IndexFlatIP` مع `normalize_L2` لبناء فهرس `faiss_index_text.flat` و `faiss_index_title.flat`.
- 11.2. التفعيل: عند تفعيل `use_faiss` في الطلب، يستخدم `RetrievalService` البحث عبر FAISS.
- 11.3. النتيجة: زمن البحث لـ BERT انخفض من ~15 ثانية إلى ~1.5 ثانية.

12. التحديات التي تم حلها (Challenges & Solutions):

- 12.1. بطء البحث في TF-IDF:
- كان السبب الرئيسي هو استخدام دالة `cosine_similarity`` لحساب التشابه بين استعلام المستخدم وجميع الوثائق (382,545 وثيقة) في كل مرة يُنفذ فيها بحث، مما تسبب في بطء شديد واستهلاك كبير للوقت. تم حل هذه المشكلة بتطبيق متجهات TF-IDF مسبقاً (أثناء مرحلة الفهرسة)، ثم استخدام الضرب النقطي المباشر (`query_vec.dot(matrix.T).A1``) لحساب التشابه، مما قلل زمن البحث بشكل كبير.
- 12.2. بطء البحث في BERT:
- اعتمد البحث الدلالي في البداية على المقارنة القسرية بين متجه الاستعلام وجميع متجهات الوثائق المخزنة (عدد 382,545 متجهاً)، مما تسبب في بطء شديد مع تزايد عدد الوثائق. تم حل هذه المشكلة باستخدام مكتبة FAISS لبناء فهرس متجهات سريع، مع تطبيق المتجهات مسبقاً لضمان التوافق مع خوارزمية البحث عن أقرب الجيران، مما حقق تسريعاً هائلاً في زمن الاستعلامات الدلالية.

13. تقسيم العمل (Team Work Distribution):

تم توزيع المهام بين أعضاء الفريق الأربعة على النحو التالي:

حسان محمد غنام: تولى مسؤولية المعالجة المسبقة للبيانات (Preprocessing) بما في ذلك استخراج الجذور (Stemming) وإعداد ملفات JSONL، بالإضافة إلى تنفيذ نموذج TF-IDF وتحسين الذاكرة باستخدام تقنيات (mmap، float32). كما شارك في بناء واجهة برمجة التطبيقات (Flask API)، واستقرار النظام، ونظام التقييم، وواجهة المستخدم.

عبدالرحمن يحيى حوير: قام بتنفيذ نموذج BM25 مع ضبط معاملاته (k1, b) لتحسين الأداء. كما شارك في بناء واجهة برمجة التطبيقات (Flask API)، واستقرار النظام، ونظام التقييم، وواجهة المستخدم.

غيث حمزه أبو راشد: تولى تنفيذ النموذج الدلالي (BERT) وتوليد التضمينات (Embeddings) مع فهرسة FAISS لتسريع البحث الدلالي، بالإضافة إلى إعداد قاعدة البيانات (SQLite) وبناء واجهة برمجة التطبيقات (Flask API). كما شارك في استقرار النظام، ونظام التقييم، وواجهة المستخدم.

الاء عيسى سليمان: قامت بتنفيذ النماذج الهجينة (Parallel و Serial Hybrid) مع اختبار استراتيجيات الدمج المختلفة. كما تولت تصميم وتطوير واجهة المستخدم (UI)، وتنفيذ نظام التقييم وعرض المقاييس باستخدام Chart.js والرسوم البيانية، وعرض النتائج.

المصادر والمراجع (References):

- 13.1. مجموعة البيانات (Dataset):
<https://ir-datasets.com/beir.html#beir/webis-touche2020>
- 13.2. DeepSeek: استخدام الذكاء الاصطناعي للمساعدة في كتابة الأكواد وحل المشكلات التقنية أثناء التطوير.
- 13.3. GitHub Copilot: للمساعدة في استكمال الكود وحل المشكلات التقنية أثناء التطوير.
- 13.4. المراجع التعليمية (Educational References):
- محاضرات المادة (النظرية والعملية) في مجال استرجاع المعلومات والتي غطت مفاهيم مثل:
13.4.1. الفضاء المتجهي (VSM) ومعامل TF-IDF.
13.4.2. نماذج الاحتمالات (BM25).
13.4.3. التضمينات الدلالية (BERT Embeddings).
13.4.4. تقييم الأنظمة (Precision، Recall، MAP، NDCG).
13.4.5. بناء الفهارس وقواعد البيانات.

14. روابط المشروع (Project Links):

- 14.1. GitHub:
<https://github.com/HASSANGHANNAM/IR-System>
محتويات المستودع الإضافية:
إلى جانب الأكواد المصدريّة، يحتوي المستودع على الملفات والمجلدات التالية لتسهيل تشغيل النظام واختباره:
 - ملف README.md: يحتوي على تعليمات تثبيت وتشغيل المشروع خطوة بخطوة، بالإضافة إلى شرح موجز عن النماذج والنتائج.
 - ملف requirements.txt: قائمة بجميع المكتبات المطلوبة لتشغيل النظام (Flask، numpy، scikit-learn، sentence-transformers، faiss-cpu، rank-bm25 وغيرها).
 - مجلد /logs: يحتوي على سجلات التشغيل (Logs) التي تم جمعها أثناء تطوير واختبار النظام، لتسهيل الأخطاء وتحليل الأداء.
 - ملف IR-System.postman_collection.json: ملف جاهز للاستيراد في تطبيق Postman، يحتوي على جميع طلبات API المستخدمة في النظام (/api/search، /api/queries، /api/evaluate) لتسهيل اختبار الواجهات الخلفية وتوثيقها.
- 14.2. رابط ملفات البيانات (Google Drive):
 Colab Notebooks
 IR_Project_Files
- 14.3. رابط الداتاسيت:
<https://ir-datasets.com/beir.html#beir/webis-touche2020>

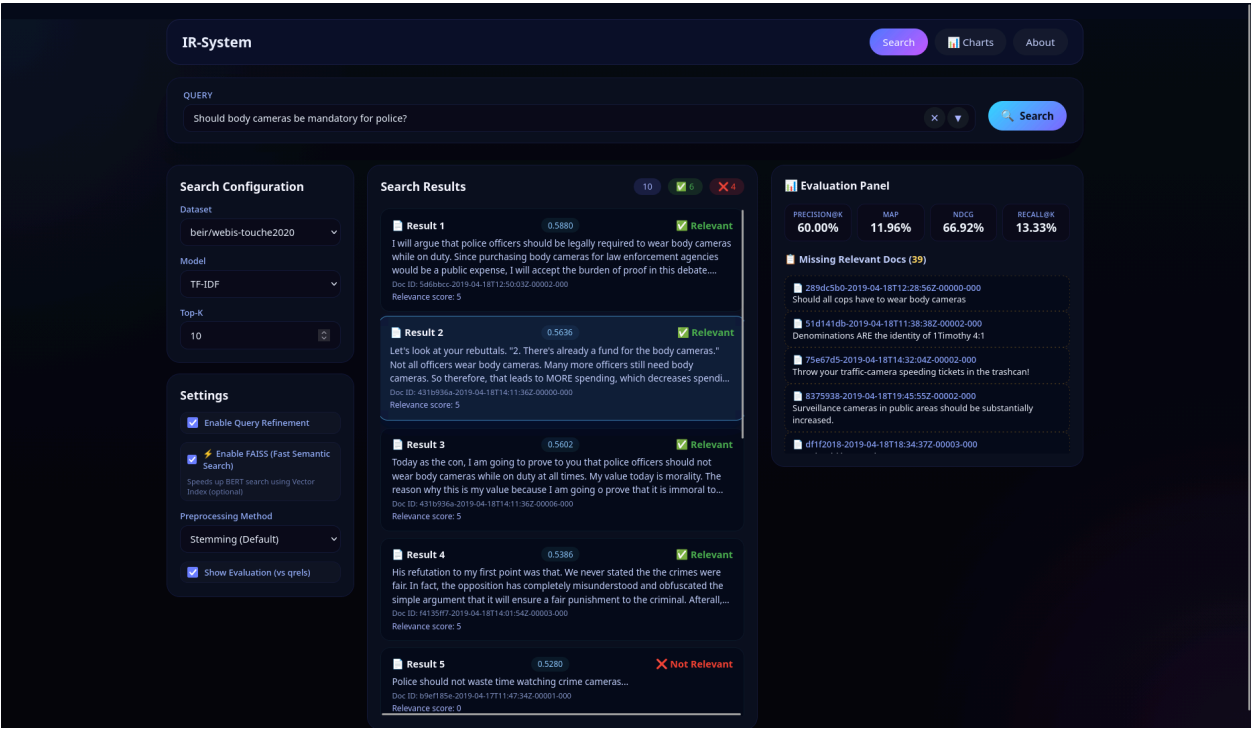
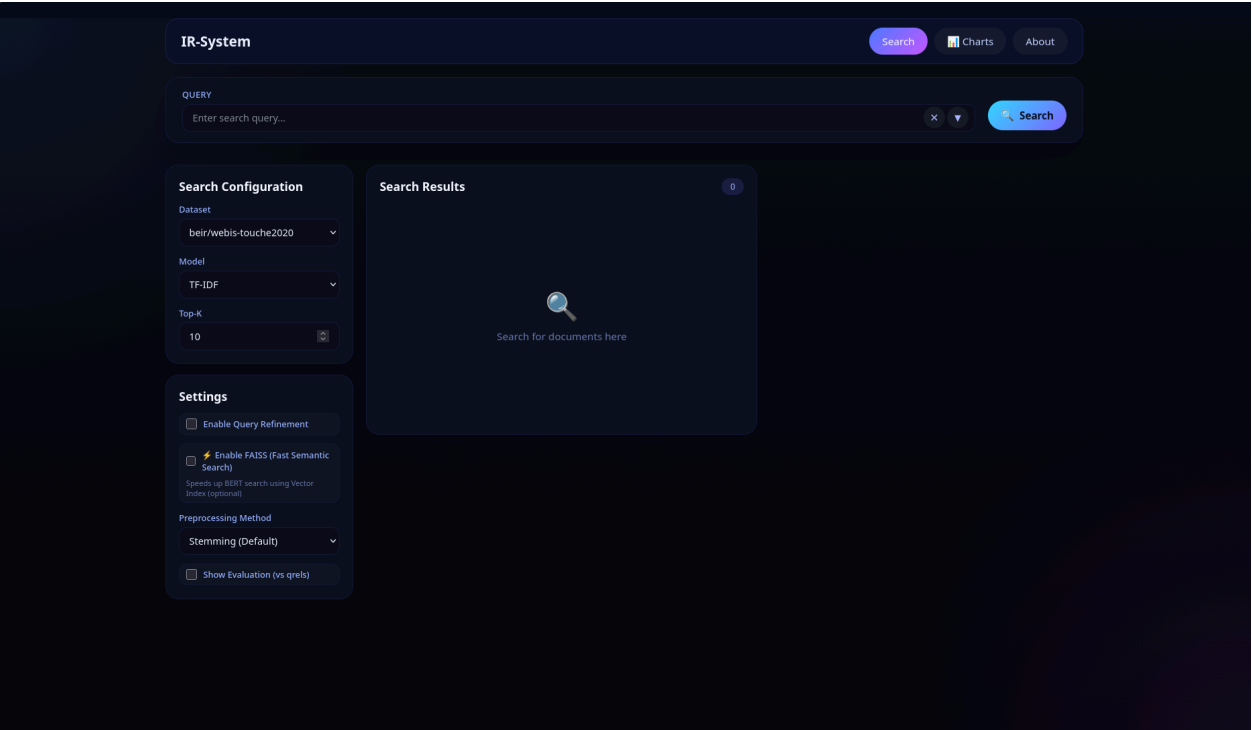
15. الخاتمة والتوصيات (Conclusion & Recommendations):

- 15.1. الخاتمة:
في هذا المشروع، تم بناء وتقييم نظام متكامل لاسترجاع المعلومات يجمع بين النماذج التقليدية والدلالية. أظهرت النتائج أن نموذج BM25 بعد ضبط معاملاته ($k1=2.0$, $b=0.60$) يتفوق بشكل حاسم على TF-IDF و BERT،

محققاً أعلى قيمة. كما تم تطبيق تحسينات متقدمة مثل الفهرسة باستخدام FAISS، والهندسة الموجهة للخدمات (SOA)، وتحسينات الذاكرة (mmap, float32)، وواجهة مستخدم غنية بالميزات مع تقييم فوري.

15.2. التوصيات النهائية:
النموذج الأساسي: اعتماد BM25 مع المعاملات المُحسَّنة ($k1=2.0$, $b=0.60$) كنموذج أساسي.
دمج الحقول: تطبيق دمج العناوين مع النص في النماذج الدلالية (مثل BERT) بوزن $\alpha=0.4$.
FAISS: تفعيله دائماً لـ BERT والهجين لتحسين زمن الاستجابة.

16. الصور ولقطات الشاشة للواجهات الرسومية.



IR-System

Search

Charts

About

QUERY

Should body cameras be mandatory for police?

X

▼

Search

Search Configuration

Dataset

beir/webis-touche2020

Model

Hybrid

Hybrid Search Configuration

Mode

Parallel

Model Weights

Set weights for each model. Use 0 to exclude a model. Weights are automatically normalized.

TF-IDF

0.33

BERT

0.33

BM25

0.34

Quick Presets

All Models Equal

The Best

How it works:

Set weight to 0 to exclude a model

Weights are automatically normalized to sum to 1

All models use the same preprocessing and ranking as individual models

Search Results

10

8

2

Result 1

9.1298

Relevant

I will argue that police officers should be legally required to wear body cameras while on duty. Since purchasing body cameras for law enforcement agencies would be a public expense, I will accept the burden of proof in this debate...

Doc ID: 5d6bacc-2019-04-18T12:50:03Z-00002-000

Relevance score: 5

Result 2

8.5095

Relevant

Today as the con, I am going to prove to you that police officers should not wear body cameras while on duty at all times. My value today is morality. The reason why this is my value because I am going o prove that it is immoral to...

Doc ID: 4316d8c-2019-04-18T14:11:36Z-00006-000

Relevance score: 5

Result 3

8.1786

Relevant

His refutation to my first point was that. We never stated the the crimes were fair. In fact, the opposition has completely misunderstood and obfuscated the simple argument that I will ensure a fair punishment to the criminal. Afteral...

Doc ID: 4413987-2019-04-18T14:01:54Z-00003-000

Relevance score: 5

Result 4

7.8820

Relevant

* This is not necessarily a round of police debate* Humanity has been on an ever eternal quest in search of felicity and safety. However, throughout mankind, there has been a series of crimes gone unsolved and detectives...

Doc ID: 4413987-2019-04-18T14:01:54Z-00005-000

Relevance score: 5

Result 5

7.1493

Relevant

Also, I'd like to apologize for the lack of formal rules and structure to this debate (I am new to the site and this is the first debate I posted); we'll have to freestyle this. Now then, addressing your points in order: --- "Now we should...

Evaluation Panel

PRECISION@K

80.00%

MAP

17.78%

NDCG

82.62%

RECALL@K

17.78%

Missing Relevant Docs (37)

289dc5b0-2019-04-18T12:28:56Z-00000-000

Should all cops have to wear body cameras

51d141db-2019-04-18T11:38:38Z-00002-000

Denominations ARE the identity of 1Timothy 4:1

75e67d5-2019-04-18T14:32:04Z-00002-000

Throw your traffic-camera speeding tickets in the trashcan!

4372938-2019-04-18T19:45:55Z-00002-000

Surveillance cameras in public areas should be substantially increased.

df1f2018-2019-04-18T18:34:37Z-00003-000

IR-System

Search

Charts

About

QUERY

Should body cameras be mandatory for police?

X

▼

Search

Search Configuration

Dataset

beir/webis-touche2020

Model

Hybrid

Hybrid Search Configuration

Mode

Parallel

Model Weights

Set weights for each model. Use 0 to exclude a model. Weights are automatically normalized.

TF-IDF

0.33

BERT

0.33

BM25

0.34

Quick Presets

All Models Equal

The Best

How it works:

Set weight to 0 to exclude a model

Weights are automatically normalized to sum to 1

All models use the same preprocessing and ranking as individual models

Search Results

10

8

2

Police officers should be legally required to wear body cameras while on duty

X

I will argue that police officers should be legally required to wear body cameras while on duty. Since purchasing body cameras for law enforcement agencies would be a public expense, I will accept the burden of proof in this debate. Sources can and should be posted in text after the arguments they are being used to support. -Power and authority should be accompanied by accountability. This is something that Sir Robert Peel recognized in his principles of policing which were developed to guide the first professional police force. One example: "The ability of the police to perform their duties is dependent upon public approval of police actions." Body cameras an oversight tool which can be used to better hold officers accountable and maintain public trust. (Source: <http://www.nwpolice.org...>) -Body cameras have the two-pronged effect of encouraging police officers to act professionally and also defusing situations with subjects (who are less inclined to behave in discrediting ways when they know they are being filmed). In pilot projects where body cameras worn by police officers have been implemented and studied, complaints against officers have dropped over 90%. (Source: <http://cjb.sagepub.com...>) -Further expanding on the point of cameras being a tool to defuse situations, they will improve officer safety as subjects will be aware that their actions will be recorded on camera and those recordings could be used as evidence against them. -While cost is sometimes cited as an argument against the implementation of police body cameras, lawsuits against police agencies for misconduct can be extremely expensive for the agencies and the government bodies that oversee them. Body cameras can capture evidence that can discredit false complaints against officers and can prevent frivolous lawsuits from succeeding or forcing governments to provide unnecessary settlements. -Individual officers themselves are protected from false accusations in the form of complaints or lawsuits if they behave professionally and appropriately and their conduct is captured through body cameras. I'm looking forward to further debating this with anyone willing to argue the contrary position.

13





انتهى التقرير